JPas

Reference Guide

Version 1.0 (Draft)

Copyright © 2017
Ashur Rafiev

Available under MIT License at:
https://github.com/ashurrafiev/JPas

Copyright © 2017 Ashur Rafiev

MIT License

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

The software is provided "as is", without warranty of any kind, express or implied, including but not limited to the warranties of merchantability, fitness for a particular purpose and noninfringement. In no event shall the authors or copyright holders be liable for any claim, damages or other liability, whether in an action of contract, tort or otherwise, arising from, out of or in connection with the software or the use or other dealings in the Software.

Contents

1 Introduction **1**
1.1 About JPascal . . . 1
1.2 Differences from Classic Pascal . . . 1
1.2.1 Program structure . . . 2
1.2.2 Variable, type, and constant declarations . . . 2
1.2.3 Unit structure . . . 2
1.2.4 Basic data types . . . 3
1.2.5 Enumerated types . . . 3
1.2.6 Records . . . 3
1.2.7 Array and record literals . . . 3
1.2.8 Sets . . . 4
1.2.9 Files . . . 4
1.2.10 For loops . . . 4
1.2.11 Pointers and memory addressing . . . 4
1.2.12 Garbage collection . . . 4
1.2.13 Functions and procedures . . . 5
1.2.14 Labels, exit and goto . . . 5

2 Language Elements **7**
2.1 Whitespace and Comments . . . 7
2.2 Keywords and Identifiers . . . 7
2.3 Constant Literals . . . 8
2.3.1 Numeric literals . . . 8
2.3.2 Character and string literals . . . 9
2.3.3 Built-in constants . . . 9
2.4 Operator symbols . . . 9

3 Program Structure **11**
3.1 Types of statements . . . 11
3.2 Scope . . . 12
3.3 Memory and Stack . . . 12

4 Variables and Constants **13**

5 Types **15**
5.1 Simple Types . . . 15
5.2 Enumerated and Subrange Types . . . 15
5.3 Array Types . . . 15

5.4 Record Types . . . 15
5.5 Pointers . . . 15

6 Expressions **17**

7 Statements **19**
7.1 Block Statements . . . 19
7.2 Declaration Statements . . . 19

8 Procedures and Functions **21**

9 Units **23**

10 Standard Procedures and Functions **25**
10.1 Execution Control . . . 25
10.1.1 Halt . . . 25
10.1.2 RunError . . . 26
10.2 Pointers . . . 26
10.2.1 New . . . 26
10.3 Console Input and Output . . . 26
10.3.1 Write and WriteLn . . . 26
10.3.2 Read and ReadLn . . . 26
10.4 Time . . . 26
10.4.1 Delay . . . 26
10.4.2 SysTime . . . 27
10.4.3 Elapsed . . . 27
10.5 Mathematical Functions . . . 27
10.5.1 Int . . . 27
10.5.2 Frac . . . 27
10.5.3 Round . . . 28
10.5.4 Sqrt . . . 28
10.5.5 Pi . . . 28
10.5.6 ArcTan . . . 28
10.5.7 Sin and Cos . . . 28
10.5.8 Exp . . . 28
10.5.9 Ln . . . 29
10.5.10 Abs . . . 29
10.5.11 Sqr . . . 29
10.5.12 Min and Max . . . 29
10.5.13 Odd . . . 29
10.6 Ordinal Functions . . . 30
10.6.1 Succ . . . 30
10.6.2 Pred . . . 30
10.6.3 Inc . . . 30
10.6.4 Dec . . . 30
10.6.5 Ord . . . 31

10.7 Strings and Characters 31
10.7.1 Chr 31
10.7.2 UpCase and LowCase 31
10.7.3 Format 31
10.7.4 Str 31
10.7.5 Val 32
10.7.6 Copy 32
10.7.7 Length 32
10.7.8 Pos 32
10.7.9 Concat 32
10.7.10 Delete 33
10.7.11 Insert 33
10.8 Arrays and Memory 33
10.8.1 Fill 33
10.8.2 Swap 33
10.9 Random Number Generator (RNG) 33
10.9.1 Randomize 33
10.9.2 Random 34

11 Graph2D Unit **35**

1. Introduction

JPas is an interpreter of Pascal-like language, called *JPascal*, running in Java Virtual Machine (JVM). It supports a variety of features, including console input and output, as well as graphical interface, mouse and keyboard events.

Why?

To do: invent a decent motivation.

1.1 About JPascal

JPascal language draws its inspiration from early Turbo Pascal versions 3.0 to 5.0, before the object oriented programming was introduced.

> **Disclaimer:** *Back in the days, Turbo Pascal was a registered trademark of Borland International, Inc., USA, and it is still a registered trademark owned by CodeGear LLC, USA. Even though, to the author's knowledge, Turbo Pascal IDE has been released to public for free, it is important to emphasise that JPascal does not copy Turbo Pascal version of Pascal programming language, as explained in this section and with more details in Section 1.2. Turbo Pascal version 5.0 is used in this manual only as a historical reference and a baseline for feature comparison, and is further referred as* classic Pascal.

...

1.2 Differences from Classic Pascal

JPascal adds new features and removes some from the classic Pascal. Many of the changes are done because of JPas environment is running in JVM. JPascal is a higher level programming language than classic Pascal. Most of the removed features are linked to a specific system architecture.

Keywords that existed in classic Pascal, but have not been added to JPascal:

absolute	**goto**	**packed**	**unit**
external	**inline**	**program**	
file	**interrupt**	**set**	

1.2.1 Program structure

Program does not start with the keyword **Program**, and units do not start with **Unit**. In fact, these are not keywords in JPascal.

Minimal "Hello world" program in classic Pascal:

```
Program Hello;
Begin
  WriteLn('Hello world!');
End.
```

In JPascal it becomes a one-liner:

```
WriteLn('Hello world!').
```

1.2.2 Variable, type, and constant declarations

There are no dedicated declaration blocks (apart from the **interface** part in a unit), all declarations can be done in-place, anywhere in the code.

1.2.3 Unit structure

Interface/implementation structure is not limited to units, however it is not very useful anywhere else. The syntax is also different, as shown below.

In classic Pascal:

```
Unit UnitName;
interface
{Public declarations.}
implementation
{Implementation and private declarations.}
Begin
{Unit initialisaton code.}
End.
```

In JPascal:

```
interface
{Public declarations.}
implementation
{Unit initialisaton code with implementation.}
end.
```

1.2.4 Basic data types

Integer numbers are represented only with Integer type, which represents 32-bit signed integer numbers. There are no Byte, Word, or LongInteger.

Real is implemented using double precision floating point numbers (64-bit).

String type does not take size as a parameter. The strings have variable length realised by Java strings.

No *packed* data types; **packed** is not a reserved keyword.

1.2.5 Enumerated types

Enumerated types are defined using square brackets [and] instead of parentheses (and). This is done for consistency with array and record literals, see Section 1.2.7.

1.2.6 Records

Variant records and tag fields are not possible.

It is not allowed to declare a pointer of type ^T before T is declared. For example, the following code is valid in classic Pascal, but will give a compilation error in JPas.

```
{Classic Pascal only!}
type
  PersonPtr = ^Person;
  Person = record
    Name: String;
    Age: Integer;
    Next : PersonPtr;
  end;
```

However, you can still use ^T within the declaration of T:

```
{In JPascal:}
type Person = record
    Name: String;
    Age: Integer;
    Next : ^Person;
  end;
```

1.2.7 Array and record literals

Array and record literals are defined using square brackets [and] (unlike regular parentheses (and) in classic Pascal. Because of this, you can use array and record literals anywhere in the code. Regular parentheses would make no distinction between single-value array and a parenthetic expression.

1.2.8 Sets

Sets are not added in this version of JPas, and probably will not be added any time soon. Even though it is a nice feature, it has a very limited application, so may not worth time investment. In the current implementation of JPas, **set** is not a reserved keyword

1.2.9 Files

Version 1.0 of JPas does not support working with files, however this functionality may be added later in one way or another. At the moment, **file** is not a keyword, and Text is not a reserved identifier.

1.2.10 For loops

Counter may be any L-value, not just a variable. You are allowed to change the counter inside the loop body. Starting and ending conditions are still evaluated only once, before the loop starts.

1.2.11 Pointers and memory addressing

JPas interpreter implements its own memory structure based on Java objects, which is different from the classical DOS memory model. It is not possible to get numerical addresses of memory items. As the result, pointer arithmetic is not possible.

Mem and Port arrays, provided in classic Pascal, do not exist in JPas. Almost all of the memory access functions and procedures are also removed.

No *absolute* access; **absolute** is not a reserved keyword.

Overlays are meaningless to modern systems, so they haven't been added to JPas.

1.2.12 Garbage collection

On the bright side, the memory management became much easier thanks to JVM's Garbage Collector (GC).

In classic Pascal, it was mandatory to call either Dispose or Mark/Release for every pointer allocated by the function New. Failing to do so would lead to memory leaks.

In JPas, disposing of unused memory is done by GC, and there are no such functions as Dispose, Mark, and Release.

```
var Q: ^Integer;
begin
  var P: ^Integer;
  New(P); {Allocate memory.}
  P^ := 8;
  Q := P;
end; {Variable P is destroyed.}

{The memory is still referenced by Q.}
WriteLn(Q^); {Prints '8'.}

Q := nil; {Clear the pointer.}
{Memory is automatically unallocated
because there are no pointers referencing it.}
```

1.2.13 Functions and procedures

No untyped arguments because absolute access is not allowed.

No **external** , **inline**, and other procedural modifiers.

No procedural types or function pointers.

Many standard functions and procedures are changed. Please refer to Section 10 for the reference on the updated versions.

1.2.14 Labels, exit and goto

Labels do not need an in-advance declaration. In JPascal, a label marks a statement.

goto statement has been deprecated in many languages. JPascal does not implement it.

Most interesting behaviour can be realised using **exit** Label statements, which exits the statement marked with the label. This always guarantees safe behaviour. It is still possible to use **exit** without a label to exit a function, procedure, or the program, like in classic Pascal.

2. Language Elements

...

2.1 Whitespace and Comments

Whitespace is any number of tabs, spaces and newline characters in any combination. A comment is any text enclosed in curly braces { and }, or between (* and *). Whitespaces and comments are ignored by the parser.

2.2 Keywords and Identifiers

The list of keywords:

and	**exit***	**nil**	**to**
array	**for**	**not**	**type**
begin	**forward**	**of**	**until**
case	**function**	**or**	**uses**
const	**if**	**procedure**	**var**
div	**implementation**	**record**	**while**
do	**in****	**repeat**	**with**
downto	**interface**	**shl**	**xor**
else	**label**	**shr**	
end	**mod**	**then**	

* **exit** was a function in classic Pascal, not a keyword.

** In the current version of JPas, **in** is a registered keyword, but the operator is not yet implemented.

Identifier is a sequence of characters including Latin letters, numbers and underscore. It must contain at least one letter and cannot start with a number. Identifiers can be used as names for variables, constants, types, functions, procedures, etc.

The list of registered identifiers:

Boolean	False	Real	True
Char	Integer	String*	

* String was a keyword in classic Pascal, now it is considered a registered identifier.

Identifiers and keywords are case-insensitive. It is considered a good programming style to write keywords in lower case and start identifiers in capital case. The exceptions are the main **Begin** and **End** keywords of the program, which are usually capitalised, and local variables with single-character names, e.g. x or y, which can be lower case.

2.3 Constant Literals

2.3.1 Numeric literals

Any sequence of digits 0–9 form a decimal integer number. Unary 'minus' operator – creates a negative number. Unary 'plus' operator + can be optionally used to emphasise positive numbers. In JPascal, integer numbers are 32-bit, hence specifying numbers larger than $\left(2^{31}-1\right)$ or smaller than $\left(-2^{31}\right)$ will produce a compilation error.

Hexadecimal integer literals are preceded with $ symbol. It is possible to specify larger numbers in hexadecimal format, but they will be cropped to 32-bit with overflow. For instance, $FFFFFFFF will produce decimal (-1).

```
{Integer numbers:}
123
–8
+8 {Same as 8}
00025 {Same as 25}
$FF {Decimal 255}
$FFFFFFFF {Decimal –1 (overflow).}
```

Real numbers have integer and fractional parts separated by . (period). Scientific notation is also allowed. Examples are shown below.

```
{Real numbers:}
0.01
3.141592
2.5E-6 {Same as 0.000025}
2.5E+6 {Same as 250000}
1E9 {1000000000.0 – one billion as a real number.}
```

> **Note:** *Scientific notation always produces real numbers, so they need to be explicitly converted to integer.*

```
Int(1E9) {Now it is an integer number.}
```

There is an implicit cast from Integer to Real type.

2.3.2 Character and string literals

Strings of characters enclosed in single quotes ' produce string or character literals. Character literals contain exactly one character. JPascal supports Unicode managed by Java. All characters are 16-bit.

```
'Hello, I am a string.'
'A' {...and this is a character.}
```

It is also possible to create single character constants from their ANSI/Unicode values by preceding a decimal or hexadecimal numeric literal with a hash # symbol.

```
{Characters from ANSI/Unicode values:}
#48 {Same as '0' (zero)}
#65 {Same as 'A' (Latin A)}
#$41 {Also 'A' (Latin A)}
#$03A9 {Capital Greek Omega Ω}
```

There is an implicit cast from Char to Sting type.

2.3.3 Built-in constants

Boolean constants True and False.
Null-pointer constant **nil** is a keyword.

2.4 Operator symbols

Operator symbols recognised by JPascal are:

;	*	=	@
,	/	<>	^
:	[	>	.
:=	]	<	..
+	(	>=	
-	)	<=	

3. Program Structure

...

JPascal program is a single statement ending with . (period) symbol. "Hello world" program in JPascal looks like this:

```
WriteLn('Hello world!').
```

WriteLn outputs information to system console, which in this case is the string literal 'Hello world!'.

Statement is defined as an executable action. Statements can be compound, internally executing more statements. For example, a block statement **begin** ... **end** is used to combine a sequence of multiple statements into one. Statements within a block statement are separated using semicolons.

```
Begin
  WriteLn('Hello world!');
  WriteLn('JPascal is here.');
  WriteLn('Bye!')
End.
```

3.1 Types of statements

This sections gives a brief overview of statements and program structuring. A detailed reference on statements can be found in the Section 7 of this guide.

Statements can be one of the following types:

- Empty statement.
- Declaration statements include variable, constant, type, function, and procedure declarations.
- Inclusion statement **uses**.
- Interface statement **interface** ... **implementation** ... **end**.
- Block statement.
- Control statements include exit, branching and loop statements.
- Assignment.

- Procedure call or expression statements.
- Scope statements include only **with** statement.

...

3.2 Scope

...

3.3 Memory and Stack

...

4. Variables and Constants

...

5. Types

...

5.1 Simple Types

5.2 Enumerated and Subrange Types

5.3 Array Types

5.4 Record Types

5.5 Pointers

...

6. Expressions

...

7. Statements

...

7.1 Block Statements

7.2 Declaration Statements

...and other.

8. Procedures and Functions

...

9. Units

...

10. Standard Procedures and Functions

...

Unlike user-defined procedures, the standard procedures and functions are processed by the pre-compiler in a special way, which opens the following advantages:

- The same function or procedure name can be used by different functions. The choice of the actual function depends on the types of the arguments (this is called *overloading*). The most notable example of overloading is type of the result being dependent on the type of the argument, like in Min, Max, or Abs functions.
- Variable number of arguments, denoted as (X, ...), means 0 or more arguments like *X*.
- Functions and procedures may take arguments of unspecified types or multiple specific types. Because of this, the section uses "invented" types that are not a part of the language but clarify what types can be accepted by the given procedure or function. These identifiers are listed below.

SomeType	Any type is accepted.
SimpleType	Only simple built-in types (Integer, Real, Boolean, Char, or String) are accepted.
Ordinal	Only ordinal types are accepted. Ordinal types are Integer, Boolean, Char, enumerated, and subrange types.

10.1 Execution Control

10.1.1 Halt

```
procedure Halt;
```

Stop the program execution end exit JPas with exit code 0 (no error). This is the same as calling **exit** statement from the main block of the program; however,

Halt can be called from anywhere in the code.

10.1.2 RunError

```
procedure RunError(Msg: String);
```

Stop the program execution with the runtime error and the specified message *Msg*. JPas exits with the code 1 (error).

10.2 Pointers

10.2.1 New

```
procedure New(var Ptr: ^SomeType);
```

10.3 Console Input and Output

10.3.1 Write and WriteLn

```
procedure Write(Value, ... : SimpleType);
procedure WriteLn(Value, ... : SimpleType);
```

10.3.2 Read and ReadLn

```
procedure Read(var Target, ... : SimpleType);
procedure ReadLn(var Target, ... : SimpleType);
procedure ReadLn(var Line: String);
```

10.4 Time

10.4.1 Delay

```
procedure Delay(Milliseconds: Integer);
```

Puts the execution into sleep for the specified number of *Milliseconds*. The exact timing is not guaranteed and the accuracy is determined by the operating system.

10.4.2 SysTime

```
function SysTime: Integer;
```

Returns system time: the number of seconds passed since 1 January 1970.

10.4.3 Elapsed

```
function Elapsed: Real;
```

Returns the number of milliseconds passed since the start of the program execution.

10.5 Mathematical Functions

10.5.1 Int

```
function Int(X: Real): Integer;
```

Returns the integer part of the real number X. This function can be used to convert Real type values to Integer.

10.5.2 Frac

```
function Frac(X: Real): Real;
```

Returns the fractional part of the real number X. The result is always a positive number.

10.5.3 Round

```
function Round(X: Real): Integer;
```

Rounds the real number X to a closest integer. This function can be used to convert Real type values to Integer.

10.5.4 Sqrt

```
function Sqrt(X: Real): Real;
```

Returns square root of X.

10.5.5 Pi

```
function Pi: Real;
```

Returns constant π.

Note: *During pre-compilation, this function is always optimised to a constant value.*

10.5.6 ArcTan

```
function ArcTan(X: Real): Real;
```

Returns arctangent of X in radians.

10.5.7 Sin and Cos

```
function Sin(X: Real): Real;
function Cos(X: Real): Real;
```

Returns the sine (or cosine) value of the angle X, where X is in radians.

10.5.8 Exp

```
function Exp(X: Real): Real;
```

10.5.9 Ln

```
function Ln(X: Real): Real;
```

10.5.10 Abs

```
function Abs(X: Real): Real;
function Abs(X: Integer): Integer;
```

Returns the absolute value of X. X can be integer or real, the result of the function will be of the same type as the argument.

10.5.11 Sqr

```
function Sqr(X: Real): Real;
function Sqr(X: Integer): Integer;
```

Returns the square of X, i.e. (X*X). X can be integer or real, the result of the function will be of the same type as the argument.

10.5.12 Min and Max

```
function Min(X, Y: Real): Real;
function Min(X, Y: Integer): Integer;
function Max(X, Y: Real): Real;
function Max(X, Y: Integer): Integer;
```

Returns the minimum (or maximum) of X and Y. X and Y can be integer or real, the result of the function will be of the same type as the arguments, or Real if their types differ.

10.5.13 Odd

```
function Odd(X: Integer): Boolean;
```

Returns True if the argument is an odd number, i.e. (X **mod** 2)<>0. Only works for integer numbers.

10.6 Ordinal Functions

10.6.1 Succ

```
function Succ(N: Ordinal): Odrinal;
```

Returns the next ordinal value for *N*. This function never goes out of range, but may overflow, as shown in the example below:

```
type Sides = [North, East, South, West];
var N: Sides = Succ(West);
WriteLn(N=North); {Prints 'true'}
```

10.6.2 Pred

```
function Pred(N: Ordinal): Odrinal;
```

Returns the previous ordinal value for *N*. This function never goes out of range, but may overflow (see Succ function).

10.6.3 Inc

```
procedure Inc(var N: Ordinal);
```

Change the value of the variable *N* to its next ordinal value. This is the same as:

```
N := Succ(N);
```

10.6.4 Dec

```
procedure Inc(var N: Ordinal);
```

Change the value of the variable *N* to its previous ordinal value. This is the same as:

```
N := Pred(N);
```

10.6.5 Ord

```
function Ord(Ordinal): Integer;
```

Returns the integer index for the ordinal N in relation to its range, starting with 0. If N is a character, the function returns its Unicode value. Integer N returns itself.

10.7 Strings and Characters

10.7.1 Chr

```
function Chr(Code: Integer): Char;
```

Returns a character for the given Unicode value.

10.7.2 UpCase and LowCase

```
function UpCase(C: Char): Char;
function UpCase(S: String): String;
function LowCase(C: Char): Char;
function LowCase(S: String): String;
```

Returns an uppercase (or lowercase) value for the given character or string of characters. The type of the function is determined by the type of its argument.

10.7.3 Format

```
function Format(Fmt: String; Value, ... : SimpleType): String;
```

To do: Java's String.format.

10.7.4 Str

```
function Str(Value: SimpleType): String;
```

Converts a value of simple type (Boolean, Integer, or Real) to a string using some default formatting. For more control, use Format function.

10.7.5 Val

```
procedure Val(S: String; var Target: Integer; var Code: Integer);
procedure Val(S: String; var Target: Real; var Code: Integer);
```

Attempts to parse a numeric value from the given string *S* assuming the radix in *Code*. Real numbers only accept radix 10 (decimal). On success, the result is put in the variable *Target*. On failure, the *Code* is reset to 0.

10.7.6 Copy

```
function Copy(S: String; StartPos, EndPos: Integer): String;
```

Returns a substring of *S* from *StartPos* to *EndPos* (inclusive). Positions are numbered starting from 1.

10.7.7 Length

```
function Length(S: String): Integer;
```

Returns the number of characters in a string.

10.7.8 Pos

```
function Pos(S, Target: String): Integer;
```

Returns the position of the first occurrence of substring *S* in the string *Target*. Positions are starting from 1. If *S* is not found, the function returns 0.

10.7.9 Concat

```
procedure Concat(var Target: String; S, ... : String);
```

The procedure concatenates a number of strings to the string variable *Target*. This is the same as:

```
Target := Target + S + ... ;
```

10.7.10 Delete

```
procedure Delete(var Target: String; StartPos, EndPos: Integer);
```

Deletes a portion from a string. Positions are starting from 1.

10.7.11 Insert

```
procedure Insert(var Target; S: String; StartPos: Integer);
```

Inserts a string into another at a given position. Positions are starting from 1.

10.8 Arrays and Memory

10.8.1 Fill

```
procedure Fill(var Target: Array of SomeType; Value: SomeType);
```

Fills the array *Target* with value *Value*.

10.8.2 Swap

```
procedure Swap(var X, Y: SomeType);
```

Swaps the values of two variables.

10.9 Random Number Generator (RNG)

10.9.1 Randomize

```
procedure Randomize;
procedure Randomize(Seed: Integer);
```

Set the seed for RNG to a specific (if given) or random (default) value.

10.9.2 Random

```
function Random: Real;
function Random(N: Integer): Integer;
```

Generate a random value. Without a parameter, the function generates a real value in the range $[0, 1)$. For the given integer argument N, the function generates an integer value in the range $[0, N)$. N must be positive.

11. Graph2D Unit

...

Index

...